전경민 (Kyungmin Jeon)

지원 직무: AI Solution Engineer (토스 AIOC) · 신입

chunkyung02@gmail.com · github.com/chunkyung02-boop
포트폴리오: chunkyung02-boop.github.io/portfolio · 블로그(Dev Archive): chunkyung02-boop.github.io/dev-archive

한 줄 소개

AI는 도구고, 판단은 제가 합니다. 빠르게 시도하고, 직접 생각하고, 작은 비용으로 쓸 수 있는 결과를 만듭니다.

AI를 연구하기보다 **만들어진 AI를 실제 서비스에서 동작하게 만드는 일**에 집중해 온 신입 엔지니어입니다.
데이터 수집 → 모델 적용 → API 서버 → 배포 → 재학습 자동화를 직접 구현했고,
팀 프로젝트에서는 외부 API·LLM 연동과 프론트엔드를 맡아 실제 배포까지 마쳤습니다.

직무 연결 요약

- **백엔드·데이터 연동:** FastAPI 추론 서버 + Supabase + Docker Compose 배포(서버 처리 보통 37ms),
  네이버·YouTube·TMAP·OpenAI API 연동
- **AI 오류 이해·보완:** DL·ML 의견 일치 시에만 즉답(정확도 99.84%),
  실전 테스트로 약점 측정, LLM 한계는 코드 가중 합산으로 보완
- **LLM 기능 구현:** OpenAI API(Vision·코스 설명 생성)를 서비스에 적용
- **반복 작업 구조화:** 수집 툴 3종(30초 → 0.6초),
  재학습→채점→교체 판단 자동화(1회 21분)
- **협업·프론트엔드:** 팀 3명 PR 협업·문서화,
  React·TypeScript PWA 구현 및 Vercel 배포

프로젝트

1. 티니핑 판별기 — 사진 한 장으로 티니핑 68종 이름 찾기

2026.09.09 ~ 2026.09.30 · 팀 2명 · AI 에이전트 트랙

`PyTorch` `Hugging Face(SigLIP2)` `scikit-learn(LDA · kNN)` `FastAPI` `Docker Compose` `Supabase` `Kaggle`

- **문제 정의:** 지식iN·카페·블로그 질문 글 **877건**을 수집·비교해 "이름 질문" 비율이 포켓몬의 **2배**(5.07% vs 2.48%)
  임을 확인
- **반복 작업 개선:** 수집 툴(캡처 저장기·영상 프레임 추출기·검색 수집기)을 만들어
  사진 1장당 수집 시간을 **약 30초 → 0.6초**로 단축
- **데이터 품질:** 영상 149편에서 후보 30,682장을 뽑아
  1,970장만 검수·채택,
  캐릭터별 사진 수 차이를
  3.4배 → 1.7배로 완화
- **모델:** 논문 4편(CLIP·ImageNet-R·SigLIP·SigLIP2)을 비교해 SigLIP2 선정·미세조정,
  68종 시험 1,360장 **정확도 98.8%**
- **신뢰도 설계:** DL 확신도와 ML(LDA+kNN) 의견이 같을 때만 바로 답하게 해
  즉답 1,266장의 **정확도 99.84%** 확보 (나머지는 후보 3개 제시)
- **현장 검증:** 실제 장난감 사진 800장 × 5조건 실전 테스트로 **94.0%** 측정,
  약점을 숨기지 않고 공개
- **운영 자동화:** 사용자 정답 수집 → Kaggle T4 재학습 → 채점 → 교체 판단을 **자동화(1회 21분)**,
  새 모델이 기준 미달이면 교체하지 않도록 구현
- **배포:** 앱·추론 서버·대시보드를 Docker Compose로 배포 (기동 7초, 서버 처리 보통 37ms)

2. NOLDA — AI 여가 코스 설계 서비스 (마포구)

2026.08 ~ 2026.09.19 · 팀 3명 · 담당: 프론트엔드 + 취향 분석 연동

`React(Vite · PWA)` `TypeScript` `네이버 지도 · TMAP` `YouTube Data API` `OpenAI API` `Vercel` `GitHub PR`

- 청년 여가 소비 데이터 분석 결과를 서비스로 옮겨, 예산·시간·위치·취향으로 시간순 코스를 설계하는 **모바일 웹앱(PWA)**을 팀으로 개발 (내 커밋 75건)
- 직선거리 기반 이동 시간이 실제와 달라 **TMAP 보행자 경로**로 교체,
  실제 도보 경로선과 구간별 보기 구현
- 프롬프트에 "사진 6 : 유튜브 4"를 적어도 사진 2장이 유튜브 200개를 이기는 문제를 발견,
  코드에서 가중 합산(0.6 : 0.4)하고
  LLM은 설명만 쓰도록 구조 변경 (LLM이 틀리는 지점을 코드로 보완)
- 소셜 로그인 복귀 시 상태가 초기화되던 문제를 해결(로그인 30일 유지),
  저장·공유·실시간 길 안내 구현
- 구현 지침서 2종과 서비스 흐름 문서를 작성해 팀 문서화,
  팀원이 구성한 배포 환경(Vercel·Render)에 PR로 통합

배포된 앱: nolda-space.vercel.app · 팀 저장소: github.com/nolda-app/nolda

3. AI 활용 개발 도구 · 아카이브 (개인)

`Claude Code` `HTML/JavaScript` `Electron`

- 프로젝트별 자료·고민·결과·작업 기록을 하나의 프로젝트 라이브러리(HTML)로 정리 (고민 기록 66건, 작업 기록 55건)
- 모델 도감, Dev Archive, 이력서 작업대(Word/PDF 저장), Claude Code 데스크톱 펫(Electron) 제작

보유 기술

- **백엔드·배포:** Python, FastAPI, Docker / Docker Compose, Supabase(PostgreSQL), 외부 API 연동
- **ML/DL:** PyTorch, Hugging Face Transformers(SigLIP2 미세조정), scikit-learn(LDA·kNN), TensorBoard, Grad-CAM
- **LLM 활용:** OpenAI API(Vision·코스 생성), Gemini API(상품 태그 분류), AI 에이전트 데이터 선별, Claude Code
- **프론트엔드:** React, TypeScript, Vite(PWA)
- **데이터·협업:** OpenCV, yt-dlp, 네이버 검색 API, EDA / Git·GitHub(PR·브랜치)

교육

- POTENUP AI 4기 · AI 에이전트 트랙 | 2026 ~ 진행 중

지원 동기

토스 AIOC가 "전화로 하던 일을 AI가 하게 만든다"는 목표를 가졌다는 점에 끌렸습니다. 저는 모델을 새로 만드는 것보다, **AI가 어디서 틀리는지 측정하고 안전장치를 붙여 실제로 쓸 수 있게 만드는 일**을 해왔습니다. 통화(ASR·TTS) 시스템 경험은 아직 없지만, 처음 접하는 기술(SigLIP2, TMAP, PWA)을 빠르게 학습해 서비스까지 배포한 경험으로 현장 적용을 빠르게 따라가겠습니다.